1. what is Object-Oriented Programming(`OOP`)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **"objects"**, which can contain data and code that manipulates that data. It organizes software design around data, or objects, rather than functions and logic.

Key Concepts of OOP:

1. **Class**
   A blueprint or template for creating objects. It defines a set of attributes (data) and methods (functions) that the created objects will have.
2. **Object**
   An instance of a class. It represents a specific entity with state (attributes/properties) and behavior (methods/functions).
3. **Encapsulation**
   The bundling of data (attributes) and methods (functions) that operate on the data into a single unit or class. It also restricts direct access to some of the object's components, which is called data hiding.
4. **Inheritance**
   A mechanism where a new class (child/subclass) inherits attributes and behaviors (methods) from an existing class (parent/superclass), allowing code reuse and the creation of hierarchical relationships.
5. **Polymorphism**
   The ability of different classes to be treated as instances of the same class through a common interface. Typically, it means methods in different classes can have the same name but behave differently.
6. **Abstraction**
   Hiding complex implementation details and showing only the necessary features of an object.

Example in Python:

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass
```

```python
class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

dog = Dog("Buddy")
cat = Cat("Whiskers")

print(dog.name, "says", dog.speak())  # Buddy says Woof!
print(cat.name, "says", cat.speak())  # Whiskers says Meow!
```

---

2. what are the main pillars of `OOP` ?

The main pillars of Object-Oriented Programming (OOP) are **four** fundamental principles that form the foundation of this paradigm. They are:

1. Encapsulation

- Bundling data (attributes) and methods (functions) that operate on the data into a single unit, typically a class.
- It restricts direct access to some of the object's components, which helps protect the internal state and maintain integrity.
- Often implemented using access modifiers like private, protected, and public.

2. Inheritance

- Mechanism by which a new class (child or subclass) derives properties and behaviors (methods) from an existing class (parent or superclass).
- Promotes code reuse and establishes a natural hierarchical relationship between classes.

3. Polymorphism

- The ability of different classes to respond to the same method call in different ways.
- It allows one interface to be used for a general class of actions, with specific behavior determined by the exact nature of the situation (or class).
- Commonly achieved via method overriding (in subclasses) and method overloading.

4. Abstraction

- The process of hiding complex implementation details and exposing only the necessary parts of an object or system.
- Helps reduce complexity and increase efficiency by allowing the user to interact with the object at a higher level without worrying about the intricate inner workings.

Quick summary:

Pillar	Description
Encapsulation	Hides data and restricts access to it
Inheritance	Derives new classes from existing ones
Polymorphism	Same interface, different underlying forms
Abstraction	Exposes only essential features, hides details

3. what is a class in python?

A **class** in Python is like a blueprint or template for creating **objects**. It defines a set of attributes (variables) and methods (functions) that the objects created from the class will have.

Key points about a Python class:

- It encapsulates data (attributes) and behavior (methods).
- You can create multiple objects (instances) from the same class, each with its own unique data.
- Classes support the core OOP concepts like inheritance, encapsulation, and polymorphism.

Basic syntax of a class in Python:

```
class ClassName:
    def __init__(self, attribute1, attribute2):
        self.attribute1 = attribute1
        self.attribute2 = attribute2

    def method1(self):
```

```
        # method body
        print(f"Attribute1 is {self.attribute1}")
```

- `class` keyword defines a new class.
- `__init__` is a special method called a **constructor**; it runs automatically when an object is created.
- `self` refers to the instance of the class (the object itself).

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name  # attribute
        self.age = age    # attribute

    def bark(self):
        print(f"{self.name} says Woof!")

# Creating objects (instances) of the class
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)

dog1.bark()  # Output: Buddy says Woof!
dog2.bark()  # Output: Lucy says Woof!
```

Here:

- `Dog` is the class.
- `dog1` and `dog2` are two instances (objects) of the `Dog` class.
- Each object has its own `name` and `age` attributes.
- Both objects can use the `bark()` method.

---

4. what is an object in python?

An **object** in Python is an **instance of a class**. It represents a specific entity that holds data (attributes) and can perform actions (methods) defined by its class.

Key points about objects:

- Objects are created from classes.

- Each object has its own unique set of attribute values.
- Objects interact with the world by using their methods.
- Everything in Python is an object (even simple things like numbers and strings!).

Example:

Using the earlier `Dog` class:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Creating an object (instance) of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)  # Output: Buddy
print(dog1.age)   # Output: 3

dog1.bark()       # Output: Buddy says Woof!
```

Here:

- `dog1` is an **object** of the `Dog` class.
- It has attributes `name` and `age`.
- It can use the method `bark()`.

In simple terms:

- **Class** = blueprint or recipe.
- **Object** = the actual cake made from that recipe.

---

5. what is `__init__` in python?

`__init__` in Python is a **special method** called the **constructor**. It's automatically called when you create a new object (an instance) of a class.

What does `__init__` do?

- It **initializes** the newly created object.
- It allows you to set initial values for the object's attributes (properties).
- Helps customize objects when they are created.

How it looks:

```
class MyClass:
    def __init__(self, value1, value2):
        self.value1 = value1
        self.value2 = value2
```

- `self` refers to the object being created.
- `value1` and `value2` are parameters passed when creating the object.
- Inside `__init__`, you assign these parameters to the object's attributes (`self.value1`, `self.value2`).

Example:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
print(dog1.name)  # Output: Buddy
print(dog1.age)   # Output: 3
```

Here, when `dog1` is created, `__init__` runs automatically, setting `dog1`'s `name` to `"Buddy"` and `age` to `3`.

Summary:

- `__init__` is used to **initialize** new objects.
- It's called automatically when you create an instance.

- It lets you pass values to set up the object right away.

---

6. what is `self` in python?

Key points about `self`:

- It represents the **current object** calling the method.
- By convention, it is named `self`, but you can technically use any name (though it's strongly discouraged to use anything else).
- It must be the **first parameter** of instance methods in a class.
- When you call a method on an object, Python automatically passes the object as the first argument (`self`).

Example to illustrate:

```python
class Dog:
    def __init__(self, name):
        self.name = name  # 'self.name' is the attribute of the object

    def bark(self):
        print(f"{self.name} says Woof!")

dog1 = Dog("Buddy")
dog1.bark()  # Python translates this to Dog.bark(dog1)
```

- When `dog1.bark()` is called, Python internally does `Dog.bark(dog1)`.
- Inside the `bark` method, `self` refers to `dog1`, so `self.name` is `"Buddy"`.

Why do we need `self`?

`self` differentiates between **instance attributes/methods** and **local variables**. Without it, Python wouldn't know which data belongs to which object.

---

7. what is the difference between class variable and instance variable?

1. Instance Variables

- Belong to **each individual object (instance)** of a class.
- Each object has its **own copy** of the instance variables.
- Defined inside methods (usually `__init__`) using `self`.
- Used to store data unique to each object.

2. Class Variables

- Belong to the **class itself**, not to any particular instance.
- There is **only one copy shared** by all instances of that class.
- Defined directly inside the class, but **outside any methods**.
- Used to store data shared by all objects of the class.

Example to illustrate:

```
class Dog:
    # Class variable
    species = "Canis familiaris"

    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)

print(dog1.name)      # Output: Buddy (instance variable)
print(dog2.name)      # Output: Lucy (instance variable)
print(dog1.species)   # Output: Canis familiaris (class variable)
print(dog2.species)   # Output: Canis familiaris (class variable)

# Changing class variable via the class affects all instances
Dog.species = "Canine"
print(dog1.species)   # Output: Canine
print(dog2.species)   # Output: Canine

# Changing class variable via an instance creates a new instance variable
for that instance
dog1.species = "Dog"
print(dog1.species)   # Output: Dog (instance variable now)
print(dog2.species)   # Output: Canine (still class variable)
```

Summary:

Feature	Instance Variable	Class Variable
Scope	Belongs to the object (instance)	Belongs to the class itself
Access	Accessed via `self`	Accessed via class name or instance
Storage	Each object has its own copy	Shared by all instances
Defined	Inside methods (usually `__init__`)	Directly inside the class
Use case	Unique data per object	Common data shared by all objects

---

8. what is inheritance in python?

Inheritance in Python is an **OOP concept** where a new class (called the **child** or **subclass**) **inherits** attributes and methods from an existing class (called the **parent** or **superclass**). This allows you to **reuse code** and create a hierarchy of classes.

Why use inheritance?

- To **reuse code** from the parent class.
- To **extend** or **modify** behavior of the parent class.
- To create a logical relationship between classes.

Basic syntax of inheritance in Python:

```python
class ParentClass:
    def parent_method(self):
        print("This is a method from the Parent class.")

class ChildClass(ParentClass):
    def child_method(self):
        print("This is a method from the Child class.")

# Create an object of ChildClass
child = ChildClass()
```

```
child.parent_method()  # Inherited from ParentClass
child.child_method()   # Defined in ChildClass
```

Example with inheritance:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} says Woof!")

class Cat(Animal):
    def speak(self):
        print(f"{self.name} says Meow!")

dog = Dog("Buddy")
cat = Cat("Whiskers")

dog.speak()  # Output: Buddy says Woof!
cat.speak()  # Output: Whiskers says Meow!
```

- `Dog` and `Cat` **inherit** from `Animal`.
- They **override** the `speak` method to provide their own implementation (this is called **method overriding**).

Summary:

- **Inheritance** lets a child class reuse code from a parent class.
- It helps avoid code duplication.
- Child classes can add or override methods and attributes.

---

9. what types of inheritance are supported in python

Python supports several types of inheritance, allowing you to design flexible and reusable class hierarchies. Here are the main types:

---

1. Single Inheritance

A child class inherits from one parent class.

```
class Parent:
    pass

class Child(Parent):
    pass
```

---

2. Multiple Inheritance

A child class inherits from **more than one** parent class.

```
class Mother:
    pass

class Father:
    pass

class Child(Mother, Father):
    pass
```

---

3. Multilevel Inheritance

A child class inherits from a parent class, and then another class inherits from that child class.

```
class Grandparent:
    pass

class Parent(Grandparent):
    pass

class Child(Parent):
    pass
```

4. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

```
class Parent:
    pass

class Child1(Parent):
    pass

class Child2(Parent):
    pass
```

5. Hybrid Inheritance

A combination of two or more types of inheritance mentioned above.

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

Summary table:

Type	Description
Single	One child inherits from one parent
Multiple	One child inherits from multiple parents
Multilevel	Inheritance chain (grandparent → parent → child)
Hierarchical	Multiple children inherit from one parent
Hybrid	Combination of above types

10. what is method overloading

Method overloading is a concept where multiple methods have the **same name** but differ in the **number or type of parameters** within the same class. It allows you to define different behaviors for a method depending on how it is called.

How does method overloading work in other languages?

Languages like Java or C++ support method overloading natively:

```
void display(int a) { ... }
void display(int a, int b) { ... }
```

The correct method is chosen based on the arguments.

What about Python?

Python **does NOT support method overloading** in the traditional sense. If you define multiple methods with the same name in a class, the **last one overwrites the previous ones**.

How to achieve similar behavior in Python?

You can use **default arguments**, `*args`, or check types inside a single method to mimic overloading.

Example using default arguments:

```
class Example:
    def greet(self, name=None):
        if name:
            print(f"Hello, {name}!")
        else:
            print("Hello!")
```

```
obj = Example()
obj.greet()          # Output: Hello!
obj.greet("Alice")   # Output: Hello, Alice!
```

Example using `*args`:

```
class Example:
    def greet(self, *args):
        if len(args) == 0:
            print("Hello!")
        elif len(args) == 1:
            print(f"Hello, {args[0]}!")
        else:
            print("Hello, everyone!")

obj = Example()
obj.greet()                # Hello!
obj.greet("Alice")         # Hello, Alice!
obj.greet("Alice", "Bob") # Hello, everyone!
```

Summary:

- Python **does not support method overloading** like Java or C++.
- Use **default parameters**, `*args`, or **type checking** inside methods to simulate overloading behavior.

---

11. what is method overriding

Method overriding is an OOP concept where a **child class provides its own version of a method** that is already defined in its **parent class**. When the method is called on an object of the child class, the child's version **replaces** (or overrides) the parent's version.

Key points about method overriding:

- Occurs in **inheritance** scenarios.
- Allows a subclass to **modify or extend** the behavior of a method from the parent class.

- The method in the child class has the **same name, same parameters** as in the parent class.
- Supports **runtime polymorphism** — the method that gets called depends on the object type at runtime.

Example:

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):  # Overriding the speak method
        print("Dog says Woof!")

animal = Animal()
dog = Dog()

animal.speak()  # Output: Animal makes a sound
dog.speak()     # Output: Dog says Woof!
```

Here, `Dog` overrides the `speak` method of `Animal`.

Why is method overriding useful?

- To provide **specific behavior** for subclasses.
- To implement **polymorphism**, allowing the same method call to do different things based on the object.

---

2. what is `polymorphism` in python

Polymorphism in Python (and OOP in general) means **"many forms"** — it allows objects of different classes to be treated through the same interface, usually by sharing method names, but each class implements those methods differently.

What does that mean practically?

- Different classes can have methods with the **same name**.
- When you call that method on an object, the version corresponding to that object's class is executed.
- This enables writing flexible and extensible code.

Example of `Polymorphism`:

```
class Dog:
    def speak(self):
        print("Woof!")

class Cat:
    def speak(self):
        print("Meow!")

def make_animal_speak(animal):
    animal.speak()

dog = Dog()
cat = Cat()

make_animal_speak(dog)  # Output: Woof!
make_animal_speak(cat)  # Output: Meow!
```

Here, both `Dog` and `Cat` have a `speak()` method, but their implementations differ. The function `make_animal_speak` calls `speak()` without worrying about the exact animal type.

Types of `Polymorphism` in Python:

1. **Duck Typing:**
   "If it looks like a duck and quacks like a duck, it's a duck."
   Python cares about whether an object has the right methods, not the object's type.
2. **Operator Overloading:**
   Using operators like `+`, `-` with different types in custom ways.
3. **Method Overriding:**
   `Subclasses` providing specific implementations of methods defined in parent classes (we talked about this).

Summary:

- `Polymorphism` means **many forms** through a common interface.
- Allows different object types to be used interchangeably if they implement the expected methods.
- Enhances flexibility and extensibility in code design.

What is Operator Overloading?

Operator overloading means giving **custom behavior** to Python's built-in operators (`+` , `-` , `*` , etc.) for objects of your own classes.

This allows your objects to behave like built-in types with respect to operators.

How to overload operators in Python?

You do this by defining **special methods** (also called **magic methods**) in your class. These methods have names surrounded by double underscores, like `__add__` , `__sub__` , `__mul__` , etc.

Example: Overloading the `+` operator

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Point({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)
p3 = p1 + p2  # Calls p1.__add__(p2)

print(p3)  # Output: Point(6, 8)
```

Common operator overload methods:

Operator	Method Name	Description
`+`	`__add__`	Addition
`-`	`__sub__`	Subtraction
`*`	`__mul__`	Multiplication
`/`	`__truediv__`	Division (true division)
`//`	`__floordiv__`	Floor division
`%`	`__mod__`	Modulus
`**`	`__pow__`	Exponentiation
`==`	`__eq__`	Equality check
`<`	`__lt__`	Less than
`>`	`__gt__`	Greater than

Why use operator overloading?

- Makes your custom objects behave like built-in types.
- Makes code more readable and intuitive.
- Enables natural syntax for operations on objects.

---

12. what is `encapsulation` in python

Encapsulation in Python (and OOP in general) is the concept of **bundling data (attributes) and methods (functions) that operate on that data into a single unit**, i.e., a class. It also means **restricting direct access to some of the object's components** to protect the integrity of the data.

Why encapsulation?

- It **hides the internal state** of an object and only exposes a controlled interface.
- Helps **prevent accidental modification** of data.
- Makes the code easier to maintain and less error-prone.

How is encapsulation done in Python?

Python doesn't enforce strict access modifiers like `private` or `protected` found in other languages, but it uses naming conventions:

Access Level	Naming Convention	Meaning
Public	`variable` or `method`	Accessible from anywhere
Protected (convention)	`_variable` or `_method`	Intended for internal use (convention only)
Private (name mangling)	`__variable` or `__method`	Not easily accessible outside the class (name mangling)

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name        # public attribute
        self._age = age         # protected attribute (convention)
        self.__salary = 50000   # private attribute (name mangling)

    def get_salary(self):
        return self.__salary

    def set_salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

p = Person("Alice", 30)
print(p.name)        # Accessible
print(p._age)        # Accessible but should be treated as 'protected'
# print(p.__salary)  # Error! Attribute not accessible directly

print(p.get_salary())  # Accessing private attribute via method

p.set_salary(60000)    # Updating salary safely
print(p.get_salary())
```

Summary:

- **Encapsulation** = bundling data & methods, hiding internal details.
- Use **naming conventions** to indicate protected/private attributes.
- Use **getter/setter methods** to access or modify private data safely.

---

13. what is abstraction in python

Abstraction in Python (and OOP in general) is the concept of **hiding the complex implementation details** and showing only the essential features of an object. It helps reduce complexity and allows the user to focus on what an object does instead of how it does it.

Why abstraction?

- To **hide unnecessary details** from the user.
- To provide a **simple interface**.
- To separate the **what** from the **how**.

How is abstraction achieved in Python?

1. **Using abstract classes and methods** with the help of the `abc` module.
2. By designing classes that expose only the necessary methods and hide the internal workings.

Abstract Classes and Methods

- An **abstract class** cannot be instantiated directly.
- It can define **abstract methods** which must be implemented by its subclasses.
- Helps enforce that certain methods are implemented in subclasses.

Example:

```
from abc import ABC, abstractmethod
```

```python
class Animal(ABC):

    @abstractmethod
    def speak(self):
        pass  # Abstract method; subclasses must implement

class Dog(Animal):
    def speak(self):
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")

# animal = Animal()  # Error! Can't instantiate abstract class

dog = Dog()
dog.speak()  # Output: Woof!

cat = Cat()
cat.speak()  # Output: Meow!
```

Summary:

- **Abstraction** hides internal details and shows only essential features.
- Use **abstract classes and methods** to define a blueprint for `subclasses`.
- Enforces a common interface for different `subclasses`.

---

14. what are `@classmethod` and `@staticmethod`

---

1. `@classmethod`

- A **class method** receives the **class itself** as the first argument, conventionally named `cls`.
- It can access or modify class state that applies across all instances.
- It **cannot access instance-specific data** (i.e., `self`).
- Often used for factory methods or methods that affect the class as a whole.

Example:

```python
class MyClass:
    count = 0  # class variable

    def __init__(self):
        MyClass.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

obj1 = MyClass()
obj2 = MyClass()

print(MyClass.get_count())  # Output: 2
```

---

2. `@staticmethod`

- A **static method** does **not receive an implicit first argument** (neither `self` nor `cls`).
- It behaves like a **regular function**, but it lives inside the class's namespace.
- It **cannot access or modify instance or class state**.
- Used for utility functions related to the class but not needing instance or class info.

Example:

```python
class MathHelper:
    @staticmethod
    def add(x, y):
        return x + y

print(MathHelper.add(5, 3))  # Output: 8
```

Summary:

Feature	`@classmethod`	`@staticmethod`
First argument	`cls` (class)	None
Can access	Class variables/methods	Neither class nor instance data
Use case	Factory methods, class-wide logic	Utility/helper methods
Called by	Class or instance	Class or instance

15. what is multiple inheritance

Multiple inheritance is a feature in Python (and some other object-oriented languages) where a **class can inherit from more than one parent class**. This means a child class can inherit attributes and methods from multiple classes, combining their functionality.

Why use multiple inheritance?

- To combine features from different classes.
- To promote code reuse across multiple classes.
- To model more complex relationships.

Syntax:

```
class Parent1:
    def method1(self):
        print("Method from Parent1")

class Parent2:
    def method2(self):
        print("Method from Parent2")

class Child(Parent1, Parent2):
    pass

c = Child()
c.method1()  # Output: Method from Parent1
c.method2()  # Output: Method from Parent2
```

Here, `Child` inherits from both `Parent1` and `Parent2`, so it can use methods from both.

Important points:

- Python uses **Method Resolution Order (MRO)** to decide which method to call if multiple parents have methods with the same name.
- You can check MRO with:

```
print(Child.mro())
```

Example with method overriding and MRO:

```
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    def greet(self):
        print("Hello from B")

class C(A):
    def greet(self):
        print("Hello from C")

class D(B, C):
    pass

d = D()
d.greet()  # Output: Hello from B (because B is before C in inheritance
list)

print(D.mro())
```

Summary:

- Multiple inheritance means a class inherits from multiple parent classes.
- Useful for combining behaviors.
- Python resolves method calls using `MRO`.

---

2. what is `MRO` (Method Resolution Order)

`MRO` (Method Resolution Order) in Python is the order in which Python looks for a method or attribute in a hierarchy of classes during inheritance, especially when **multiple inheritance** is involved.

Why is MRO important?

When a class inherits from multiple parent classes that might have methods or attributes with the **same name**, Python needs a rule to decide **which method to call**.

How does Python determine MRO?

- Python uses the **C3 linearization algorithm** (also called **C3 superclass linearization**).
- It creates a linear order of classes to follow when looking up methods or attributes.
- This order ensures:
  - A class always appears before its parents.
  - The order of parent classes is preserved.

Checking MRO in Python

You can see the MRO of a class by:

```
print(ClassName.mro())
```

or

```
print(ClassName.__mro__)
```

Example:

```
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    def greet(self):
        print("Hello from B")

class C(A):
    def greet(self):
        print("Hello from C")

class D(B, C):
    pass
```

```
d = D()
d.greet()  # Output: Hello from B

print(D.mro())
```

Output of `D.mro()`:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

This means Python looks for methods in the order: `D` → `B` → `C` → `A` → `object`.

Summary:

- **MRO** defines the order Python follows to find methods in a hierarchy.
- Important in **multiple inheritance** to resolve method conflicts.
- Use `ClassName.mro()` or `ClassName.__mro__` to view it.

---

explain how **`MRO`** helps solve the famous **diamond problem** in multiple inheritance.

What is the diamond problem?

The diamond problem occurs when you have a class hierarchy like this:

```
      A
     / \
    B   C
     \ /
      D
```

- Class `D` inherits from both `B` and `C`.
- Both `B` and `C` inherit from `A`.
- If `A` has a method, and `B` and `C` don't override it, then when you call that method on `D`, which path should Python follow — through `B` or `C`?

Without a clear rule, this ambiguity causes the diamond problem.

How Python solves it with MRO

Python uses the **C3 linearization algorithm** to create a **Method Resolution Order (MRO)** that:

- Ensures each class appears **before** its parents.
- Preserves the order of base classes as declared.
- Visits each class **only once** to avoid repeated calls to the same method.

This way, Python **linearizes** the diamond-shaped hierarchy into a single, well-defined method lookup path.

Example:

```
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    pass

class C(A):
    def greet(self):
        print("Hello from C")

class D(B, C):
    pass

d = D()
d.greet()  # Output: Hello from C

print(D.mro())
```

Output of `D.mro()`:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

- When `d.greet()` is called, Python looks for `greet` in `D` → `B` → `C` → `A`.
- Since `B` doesn't have `greet`, it moves to `C` which **does** have `greet`, so it uses `C`'s method.
- This resolves ambiguity in a consistent way.

Why is this useful?

- **Avoids duplicate calls** to the same method.
- Maintains a predictable, consistent order of method calls.
- Prevents confusion and errors in complex multiple inheritance scenarios.

---

2. what is operator overloading

Operator overloading in Python is a way to give **custom behavior** to built-in operators (like `+`, `-`, `*`, etc.) for objects of your own classes. This allows your objects to interact using operators just like built-in types do.

How does it work?

You define special methods in your class, known as **magic methods** or **`dunder` methods** (because they start and end with double underscores), to specify how operators behave.

Common operator methods:

Operator	Method	Description
`+`	`__add__`	Addition
`-`	`__sub__`	Subtraction
`*`	`__mul__`	Multiplication
`/`	`__truediv__`	Division
`==`	`__eq__`	Equality comparison
`<`	`__lt__`	Less than
`>`	`__gt__`	Greater than

Example: Overloading `+` operator

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```python
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Point({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)
p3 = p1 + p2  # Calls p1.__add__(p2)

print(p3)  # Output: Point(6, 8)
```

Why use operator overloading?

- To make your custom objects behave like built-in types.
- To make code more intuitive and readable.
- To implement natural syntax for operations on objects.

---

16. what is data `hidding`

Data hiding is an important concept in Object-Oriented Programming that means **restricting direct access to some of an object's data (attributes)** to protect it from unwanted or accidental modification.

What is data hiding?

- It **prevents external code** from directly accessing or changing sensitive data inside an object.
- Instead, data is accessed or modified through **methods (getters/setters)** that control and validate changes.
- This helps maintain the **integrity and security** of the data.

How is data hiding done in Python?

Python doesn't enforce strict access controls like `private` or `protected` keywords in some other languages. Instead, it relies on **naming conventions**:

Convention	Meaning
`public_var`	Public attribute, accessible anywhere
`_protected_var`	Protected attribute (by convention, for internal use only)
`__private_var`	Private attribute (name mangling to avoid direct access)

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name          # Public
        self._age = age           # Protected (convention)
        self.__salary = 50000     # Private (name mangling)

    def get_salary(self):
        return self.__salary

    def set_salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary")

p = Person("Alice", 30)
print(p.name)          # Works fine
print(p._age)          # Accessible, but should be treated as protected
# print(p.__salary)    # Error: can't access directly

print(p.get_salary())  # Access private variable safely

p.set_salary(60000)    # Update safely
print(p.get_salary())
```

Why is data hiding useful?

- Protects data from accidental corruption.
- Controls how data is accessed or changed.
- Makes debugging and maintenance easier.

Python's **properties** provide a clean and elegant way to control access to hidden (or private) data in a class.

What are properties?

- Properties allow you to **define methods that get or set an attribute value**, but access them like regular attributes.
- This lets you **encapsulate data**, adding logic when getting or setting values **without changing how you use the attribute** from outside the class.
- They act as **controlled interfaces** to private variables.

How to create a property?

You use the `@property` decorator to define a getter method.
Then, you use `@<property_name>.setter` to define the setter method.

Example:

```
class Person:
    def __init__(self, name, age):
        self._name = name         # protected attribute
        self._age = age           # protected attribute

    @property
    def age(self):
        print("Getting age")
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            print("Age cannot be negative")
        else:
            print("Setting age")
            self._age = value

p = Person("Alice", 30)

print(p.age)    # Calls the getter
p.age = 35      # Calls the setter
```

```
print(p.age)

p.age = -5      # Invalid value, setter prevents it
```

What happens here?

- `p.age` looks like accessing an attribute but calls the `getter` method.
- Assigning `p.age = 35` calls the setter method, where you can add validation or other logic.
- If you try to set an invalid value (like `-5`), the setter can reject or handle it.

Why use properties?

- To **hide internal variables** (usually prefixed with `_` or `__`).
- To **add validation or side effects** when getting or setting attributes.
- To keep a **clean, simple interface** for users of your class.

---

17. what is difference between composition and inheritance?

Both **composition** and **inheritance** are fundamental ways to **reuse code and build relationships between classes** in object-oriented programming, but they do it differently.

---

1. Inheritance

- Represents an **“is-a” relationship**.
- A child class **inherits** attributes and methods from a parent class.
- Supports **code reuse** by extending or modifying existing behavior.
- Enables **polymorphism** — child class objects can be treated as parent class objects.

Example:

```
class Animal:
    def speak(self):
        print("Animal speaks")
```

```python
class Dog(Animal):  # Dog *is an* Animal
    def speak(self):
        print("Woof!")

dog = Dog()
dog.speak()  # Output: Woof!
```

---

2. Composition

- Represents a **“has-a” relationship**.
- A class is **composed of** one or more objects of other classes as members.
- Enables building complex objects by combining simpler ones.
- Favors **flexibility** and **encapsulation** — internal objects can be replaced or changed without affecting the whole.

Example:

```python
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()  # Car *has an* Engine

    def start(self):
        self.engine.start()

car = Car()
car.start()  # Output: Engine started
```

Key differences:

Feature	Inheritance	Composition
Relationship	**is-a** (Dog is an Animal)	**has-a** (Car has an Engine)
Code reuse	Child class inherits from parent	Class uses other class objects
Coupling	More tightly coupled	Loosely coupled
Flexibility	Less flexible, harder to change hierarchy	More flexible, components can be swapped easily

Feature	Inheritance	Composition
Polymorphism	Supports polymorphism	Does not directly support polymorphism

---

When to use?

- Use **inheritance** when there is a clear hierarchical relationship and shared behavior.
- Use **composition** to build complex types from simpler components, especially when you want more flexibility and encapsulation.

---

18. what are magic(`dunder`) methods

Magic methods (also called `dunder` **methods** because they have double underscores `__` before and after their names) are special methods in Python that let you define or customize the behavior of your objects with respect to built-in operations.

What are magic methods?

- They are **predefined methods** Python looks for when performing certain operations.
- They allow you to **override or extend** the behavior of operators, built-in functions, and language constructs.
- You don't call them directly; Python calls them **implicitly**.

Examples of common magic methods:

Operation	Magic Method	What it does
Object creation	`__init__`	Initializes a new object
String representation	`__str__`	Defines what `print(obj)` shows
Addition	`__add__`	Defines behavior for `obj1 + obj2`
Equality check	`__eq__`	Defines behavior for `obj1 == obj2`
Length	`__len__`	Defines behavior for `len(obj)`
Item access	`__getitem__`	Defines behavior for `obj[key]`

Operation	Magic Method	What it does
Context manager entry	`__enter__`	Used with `with` statements
Context manager exit	`__exit__`	Used with `with` statements

Simple example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Point({self.x}, {self.y})"

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)

print(p1)        # Calls p1.__str__() → Point(1, 2)
print(p1 + p2)   # Calls p1.__add__(p2) → Point(4, 6)
```

Why are magic methods useful?

- They make your classes **integrate smoothly** with Python's syntax and built-in functions.
- Allow you to create **intuitive, readable, and Pythonic code**.
- Enable operator overloading, custom iteration, context management, and more.

---

19. what is the difference between shallow copy and deep copy

---

Understanding the difference between **shallow copy** and **deep copy** is important when working with complex objects in Python.

Shallow Copy

- Creates a **new object**, but **inserts references** to the original objects' elements.
- For objects containing other objects (like lists of lists), the **inner objects are not copied**; both the original and the copy refer to the same inner objects.
- Changes to mutable nested objects affect both copies.

Deep Copy

- Creates a **new object**, and **recursively copies all nested objects** inside it.
- The new copy is **completely independent** of the original.
- Changes to any nested objects in the copy **do not affect the original**.

Example to illustrate:

```python
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow[0][0] = 'A'
deep[1][1] = 'B'

print("Original:", original)  # Output: [['A', 2], [3, 4]]
print("Shallow:", shallow)    # Output: [['A', 2], [3, 4]]
print("Deep:", deep)          # Output: [[1, 2], [3, 'B']]
```

Explanation:

- When we change `shallow[0][0]`, it also changes `original[0][0]` because the inner lists are shared (not copied).
- When we change `deep[1][1]`, the original stays unchanged because `deep` has its own independent copy of the nested lists.

Summary:

Copy Type	Copies Outer Object?	Copies Inner Objects?	Changes Affect Original?
Shallow Copy	Yes	No	Yes, if inner objects are mutable
Deep Copy	Yes	Yes	No

---

20. what is the difference between `is` and `==`

Python, `is` and `==` are both used for comparisons, but they do very different things:

`is` operator

- Checks **identity**: whether two variables **point to the same object** in memory.
- Returns `True` if both refer to the **exact same object**.
- Used to compare **object identities**.

```
a = [1, 2, 3]
b = a
print(a is b)  # True, both point to the same list object
```

`==` operator

- Checks **equality**: whether two variables have the **same value/content**.
- Returns `True` if the values are equal (according to the class's `__eq__` method).
- Used to compare **values**.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True, values inside the lists are equal
print(a is b)  # False, they are different objects in memory
```

Summary

Operator	Checks	Result is True When
`is`	Object identity (memory address)	Both variables refer to the **same object**
`==`	Object equality (value/content)	Both variables have **equal values**

When to use which?

- Use `is` when you want to check if two variables refer to the **same object** (e.g., `None` checks: `if x is None`).
- Use `==` when you want to check if two objects have **equivalent values**.

---

2. can you instantiate and abstract class

No, you cannot instantiate an abstract class directly.

Why?

- An **abstract class** is a class that **contains one or more abstract methods**—methods declared but **not implemented**.
- It serves as a **blueprint** for other classes.
- You must **inherit from the abstract class** and **implement all its abstract methods** before you can create objects.

In Python

You create abstract classes using the `abc` module with the `ABC` class and `@abstractmethod` decorator.

Example:

```
from abc import ABC, abstractmethod

class Animal(ABC):
```

```
    @abstractmethod
    def speak(self):
        pass

# Trying to instantiate abstract class raises error
# a = Animal()  # TypeError: Can't instantiate abstract class Animal with
abstract methods speak

class Dog(Animal):
    def speak(self):
        print("Woof!")

dog = Dog()  # Works fine
dog.speak()  # Output: Woof!
```

Summary:

- Abstract classes **cannot be instantiated** because they have incomplete methods.
- You must **inherit and implement all abstract methods** in a subclass.
- Only then can you create instances of the subclass.

---

21. what is the use of `super()` in python

What is `super()` in Python?

`super()` is a built-in function that **returns a proxy object** that allows you to call methods from a **parent (or superclass)** in a class hierarchy. It's commonly used to:

- Access parent class methods from a child class.
- Avoid explicitly naming the parent class, making code more maintainable.
- Support multiple inheritance and cooperative method calls.

Why use `super()`?

- To **reuse or extend the behavior** of the parent class.
- To **avoid repeating code**.
- To **properly work with multiple inheritance** (following the MRO).

Example:

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # Call the parent __init__
        self.breed = breed

    def speak(self):
        super().speak()          # Call the parent speak()
        print(f"{self.name} says Woof!")

dog = Dog("Buddy", "Golden Retriever")
dog.speak()
```

Output:

```
Buddy makes a sound.
Buddy says Woof!
```

Summary:

- `super()` helps **call parent class methods** easily.
- Makes your code cleaner and more flexible.
- Essential in multiple inheritance scenarios to follow the MRO correctly.

Here are a few common OOP questions with concise answers that often come up in interviews or learning sessions:

Q1: What is the difference between an abstract class and an interface?

Answer:

- **Abstract Class:** Can have both abstract methods (without implementation) and concrete methods (with implementation). Supports inheritance of code.
- **Interface:** In Python, interfaces are usually implemented via abstract classes with only abstract methods. It defines a contract with no implementation.

Q2: What is method resolution order (MRO) in Python?

Answer:
MRO is the order in which Python looks for methods in a hierarchy of classes, especially important in multiple inheritance. It ensures a consistent, linear order for method lookup.

Q3: Can a Python class inherit from multiple classes?

Answer:
Yes, Python supports **multiple inheritance** where a class can inherit from multiple base classes.

Q4: What is a decorator in Python?

Answer:
A decorator is a function that **wraps another function or method** to extend or modify its behavior without changing its code.

Q5: What is the difference between class method, static method, and instance method?

Answer:

- **Instance method:** Takes `self` as the first parameter; operates on instance data.
- **Class method:** Takes `cls` as the first parameter; works with the class itself, not instance data. Decorated with `@classmethod`.
- **Static method:** Doesn't take `self` or `cls`; behaves like a regular function inside a class namespace. Decorated with `@staticmethod`.

Got it! Let's dive deep into each of those questions with detailed explanations and examples:

Q1: Abstract Class vs Interface

Abstract Class:

- Can have both **implemented methods** and **abstract methods** (methods without implementation).
- Used to provide a **common base** with some shared code and some methods that subclasses *must* implement.
- Cannot be instantiated directly.
- Supports **single and multiple inheritance**.

Interface (in Python terms):

- Python does not have a formal interface keyword like Java or C#.
- Usually represented by an **abstract class that contains only abstract methods** (no implementation).
- Defines a **contract** that subclasses must fulfill.
- Helps with **loose coupling** and designing systems with interchangeable parts.

Example:

```python
from abc import ABC, abstractmethod

# Abstract class
class Vehicle(ABC):
    def start(self):
        print("Vehicle starting...")

    @abstractmethod
    def drive(self):
        pass

class Car(Vehicle):
    def drive(self):
        print("Car driving")

# Interface-like abstract class
class Flyable(ABC):
    @abstractmethod
    def fly(self):
        pass

class Airplane(Vehicle, Flyable):
```

```
    def drive(self):
        print("Airplane taxiing")

    def fly(self):
        print("Airplane flying")

# Usage
car = Car()
car.start()  # Inherited concrete method
car.drive()

plane = Airplane()
plane.start()
plane.drive()
plane.fly()
```

Q2: Method Resolution Order (MRO)

- MRO determines the **order in which base classes are searched when looking for a method**.
- Python uses **C3 linearization** algorithm to compute MRO.
- Important for multiple inheritance to avoid ambiguity.

Check MRO example:

```
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    def greet(self):
        print("Hello from B")

class C(A):
    def greet(self):
        print("Hello from C")

class D(B, C):
    pass

print(D.mro())  # Shows the order Python uses to look for methods
d = D()
d.greet()       # Output: Hello from B
```

- `D.mro()` output: `[D, B, C, A, object]`

- When `d.greet()` is called, Python looks for `greet` in `D`, then `B`, then `C`, etc.

Q3: Multiple Inheritance in Python

- A class can inherit from **multiple classes**.
- Method calls follow the MRO.
- Allows combining functionality from several classes.

Example:

```
class Flyer:
    def fly(self):
        print("Flying")

class Swimmer:
    def swim(self):
        print("Swimming")

class Duck(Flyer, Swimmer):
    pass

d = Duck()
d.fly()
d.swim()
```

Q4: Decorators

- Functions that **modify the behavior of other functions or methods**.
- Often used for logging, access control, memoization, etc.

Example:

```
def decorator(func):
    def wrapper():
        print("Before the function call")
        func()
        print("After the function call")
    return wrapper

@decorator
def say_hello():
    print("Hello!")
```

```
say_hello()
```

Output:

```
Before the function call
Hello!
After the function call
```

Q5: Instance, Class, and Static Methods

Method Type	First Parameter	Description	Decorator
Instance	`self`	Works on instance data	None (default)
Class	`cls`	Works on class data, shared across instances	`@classmethod`
Static	None	Does not access instance or class data	`@staticmethod`

Example:

```python
class MyClass:
    def instance_method(self):
        print(f"Instance method called, self = {self}")

    @classmethod
    def class_method(cls):
        print(f"Class method called, cls = {cls}")

    @staticmethod
    def static_method():
        print("Static method called")

obj = MyClass()
obj.instance_method()  # Needs instance
MyClass.class_method()  # Called on class
MyClass.static_method()  # Called on class, no self or cls needed
```

Absolutely! Let’s go deeper into some advanced concepts related to these topics and add more examples, explanations, and practical uses.

1. Abstract Classes & Interfaces — Advanced

Multiple Abstract Methods and Partial Implementation

Abstract classes let you provide some implementation and force subclasses to implement the rest.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    def area(self):
        print("Area calculation not implemented")

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h  # Override concrete method

    def perimeter(self):
        return 2 * (self.w + self.h)

rect = Rectangle(4, 5)
print(rect.area())       # 20
print(rect.perimeter())  # 18
```

- Subclasses **must** implement `perimeter`.
- They **can override** `area` or use the default.

---

2. Method Resolution Order (MRO) — Diamond Problem

When multiple inheritance forms a diamond shape, MRO solves ambiguity.

```
class A:
    def who(self):
        print("A")

class B(A):
```

```
    def who(self):
        print("B")

class C(A):
    def who(self):
        print("C")

class D(B, C):
    pass

d = D()
d.who()  # Output: B

print(D.mro())  # [D, B, C, A, object]
```

- Python uses **C3 linearization** to determine MRO.
- Ensures consistent and predictable method lookups.

---

3. Decorators — Advanced Usage

Decorators with Arguments

```
def repeat(num_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
```

Output:

```
Hello, Alice!
Hello, Alice!
Hello, Alice!
```

Class Method Decorators for Alternative Constructors

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data_str):
        name, age = data_str.split(',')
        return cls(name, int(age))

p = Person.from_string("John,30")
print(p.name, p.age)
```

---

4. Instance, Class, Static Methods — When to Use

- **Instance Methods:** Use for operations on instance data.
- **Class Methods:** Useful for factory methods or accessing/modifying class state.
- **Static Methods:** For utility functions related to the class but not requiring instance or class data.

Example of class state:

```python
class Counter:
    count = 0  # Class variable

    def __init__(self):
        Counter.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

c1 = Counter()
c2 = Counter()
print(Counter.get_count())  # 2
```

---

5. Deep Copy vs Shallow Copy — More Edge Cases

Example with Custom Classes

```python
import copy
```

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

node1 = Node(1)
node2 = Node(2)
node1.next = node2

shallow_node = copy.copy(node1)
deep_node = copy.deepcopy(node1)

# Modifying shallow copy’s next node affects original
shallow_node.next.value = 99
print(node1.next.value)  # 99

# Modifying deep copy’s next node does not affect original
deep_node.next.value = 100
print(node1.next.value)  # Still 99
print(deep_node.next.value)  # 100
```

---

6. More on `super()`

Multiple Inheritance Example Using `super()`

```python
class A:
    def greet(self):
        print("Hello from A")
        super().greet()

class B:
    def greet(self):
        print("Hello from B")

class C(A, B):
    pass

c = C()
c.greet()
```

Output:

```
Hello from A
Hello from B
```

- `super()` in class `A` calls the next method in MRO, which is `B.greet`.
- Without `super()`, only `A.greet` would run.

---

more advanced and commonly asked OOP topics in Python, along with explanations and examples. These go beyond the basics and will strengthen your understanding significantly.

✅ 7. Duck Typing in Python

What is it?

> **"If it looks like a duck and quacks like a duck, it's a duck."**

Python is **dynamically typed** and uses **duck typing** — it cares about **behavior**, not the actual type of the object.

Example:

```
class Duck:
    def quack(self):
        print("Quack!")

class Person:
    def quack(self):
        print("I'm pretending to be a duck!")

def make_it_quack(thing):
    thing.quack()

make_it_quack(Duck())     # Output: Quack!
make_it_quack(Person())   # Output: I'm pretending to be a duck!
```

- No need for both classes to inherit from a common parent — Python only cares that `quack()` exists.

✅ 8. Custom `__str__` vs `__repr__`

Method	Purpose	Used by
`__str__`	For end-users (readable)	`print()`, `str()`
`__repr__`	For developers (debugging)	`repr()`, console

Example:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} ($${self.price})"

    def __repr__(self):
        return f"Product('{self.name}', {self.price})"

p = Product("Laptop", 999)
print(str(p))     # Laptop ($999)
print(repr(p))    # Product('Laptop', 999)
```

✅ 9. Immutable Custom Objects

Want to make your class objects **immutable** like tuples or strings? Override `__setattr__`.

```
class ImmutablePoint:
    def __init__(self, x, y):
        super().__setattr__('x', x)
        super().__setattr__('y', y)

    def __setattr__(self, key, value):
        raise AttributeError("Cannot modify immutable instance")

p = ImmutablePoint(1, 2)
print(p.x)
# p.x = 10  # Raises AttributeError
```

✅ 10. Using `__slots__` for Memory Optimization

By default, each Python object uses a dynamic dictionary (`__dict__`) to store attributes. `__slots__` tells Python **exactly which attributes are allowed**, saving memory.

```python
class Person:
    __slots__ = ['name', 'age']  # Only these are allowed

    def __init__(self, name, age):
        self.name = name
        self.age = age

p = Person("Alice", 30)
# p.address = "USA"  # Error: attribute not allowed
```

- Useful in memory-constrained applications or when creating many objects.

✅ 11. Property with Validation (Getter/Setter Alternative)

Instead of manually creating `get_name()` and `set_name()`, use `@property`.

```python
class User:
    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age cannot be negative")
        self._age = value

u = User(25)
u.age = 30     # Valid
# u.age = -5   # Raises ValueError
print(u.age)
```

✅ 12. Metaclasses (Advanced OOP)

A **metaclass** is a class of a class. While classes create objects, **metaclasses create classes**.

> You rarely need this, but it's powerful for building frameworks and enforcing class rules.

```python
class Meta(type):
    def __new__(cls, name, bases, dct):
        print(f"Creating class: {name}")
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass

# Output: Creating class: MyClass
```

- Use cases: ORM frameworks (like Django), automatic registration, validation, etc.

✅ 13. Callable Objects with `__call__`

If you define `__call__`, your object can be used like a function.

```python
class Greeter:
    def __init__(self, name):
        self.name = name

    def __call__(self):
        print(f"Hello, {self.name}!")

g = Greeter("Alice")
g()  # Output: Hello, Alice!
```

✅ 14. Object Lifecycle: `__new__` and `__del__`

- `__new__`: Controls object creation.
- `__del__`: Destructor — called when an object is about to be destroyed.

```python
class Lifecycle:
    def __new__(cls):
        print("Creating object")
        return super().__new__(cls)

    def __init__(self):
        print("Initializing object")

    def __del__(self):
        print("Destroying object")
```

```
obj = Lifecycle()
del obj  # May not run immediately (depends on garbage collection)
```

✅ 15. Enums in Python (OOP alternative to constants)

```
from enum import Enum

class Status(Enum):
    PENDING = 1
    APPROVED = 2
    REJECTED = 3

print(Status.APPROVED.name)  # APPROVED
print(Status.APPROVED.value) # 2
```

✅ Want more?

I can go further with:

- OOP **design patterns** in Python (e.g., Singleton, Factory, Strategy)
- Writing **testable** object-oriented code
- Real-world mini-projects using OOP

what is use of super()

🧠 What is `super()` in Python?

➡️ `super()` is a built-in function used in **object-oriented programming** to **call methods (or access attributes) of a parent class** from a child class.

It is mostly used when you **override a method** in a subclass but still want to **reuse the parent class functionality**.

✅ Definition

> `super()` returns a temporary object of the **parent class**, allowing you to **call its methods or constructors** from the child class.

📘 Basic Example: Calling Parent Constructor

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)   # Call Parent's constructor
        self.age = age

c = Child("Atiar", 21)
print(c.name)  # Atiar
print(c.age)   # 21
```

✅ Here:

- `super().__init__(name)` calls the parent class `Parent` constructor.
- You don't need to call `Parent.__init__(self, name)` manually.

📘 Example: Calling Parent Method

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet(self):
        super().greet()   # Call Parent's greet
        print("Hello from Child")

c = Child()
c.greet()
```

Output:

```
Hello from Parent
Hello from Child
```

⚙️ Why use `super()`?

1. ✅ **Avoid explicitly naming the parent class** → makes code cleaner.

2. ✅ **Useful in multiple inheritance** → `super()` follows **method resolution order (MRO)**.
3. ✅ **Reusability** → reuse parent methods without rewriting code.
4. ✅ **Maintainability** → changes in parent class automatically reflect in child.

💡 Note

- `super()` is mostly used inside **class methods**.
- For multiple inheritance, Python resolves the order automatically (MRO).